SahiReport — Security & Launch Readiness Report

Second pass · Flutter client + Supabase backend · field-reporting application

Repository: **SahiReport** · Commit: **22cb38d** · Version: 1.0.0+1 · Review date: 31 October 2026 · Method: static review of `lib/`, Edge Functions, platform config, git history; automated scan attempted and blocked (see §5)

Verdict: NOT ready for public launch.

Six findings independently block release. The payment flow is more broken than the first pass reported: there is no Razorpay order, so server-side signature verification is impossible even if written today, and the amount is chosen by the client. Separately, "Delete account" destroys the login while leaving every submission and photo in place, then silently swallows the failures. Do not switch Razorpay to a live key, and do not publish the Data Safety form, until C1/N1 and N2 are closed.

6	10	8	2	26
CRITICAL	HIGH	MEDIUM	LOW	TOTAL FINDINGS

ID	FINDING	SEVERITY	STATUS	EVIDENCE
C1	**Pro entitlement written by the client after payment** The app sets `plan_type`, `pro_project_slots` and `pro_expires_at` itself. A modified client grants unlimited Pro for free.	CRITICAL	UPGRADED	`billing_screen.dart:67-72`
N1	**No Razorpay order and a client-chosen amount** Checkout opens without `order_id`, so no `razorpay_signature` is ever issued and verification is impossible. `amount` is computed on the device.	CRITICAL	NEW	`billing_screen.dart:28, 111-122`
N2	**Account deletion deletes the login and keeps the data** `supabase-js` never throws on a failed delete, so the `catch` blocks cannot fire. All five deletes can fail silently, then the auth user is removed anyway.	CRITICAL	NEW	`delete-account/index.ts:72-92`
C2	**Release builds signed with the debug keystore** The debug key is public. Play rejection risk; third parties can sign an "update" Android accepts.	CRITICAL	CONFIRMED	`android/app/build.gradle.kts:33`

ID	FINDING	SEVERITY	STATUS	EVIDENCE
C3	**RLS policies absent from the repository** Every code-user query reaches PostgREST as `anon` with a key shipped in the APK. RLS is the only boundary and it cannot be audited from source.	CRITICAL	CONFIRMED	no `supabase/migrations/`, no `config.toml`
C4	**Public bucket with unauthenticated uploads** Photos upload with no session and are read through `getPublicUrl()`. Anyone with the anon key can write, and every URL is permanent.	CRITICAL	CONFIRMED	`submission_repository.dart:167-174, 222-230`
C5	**Guessable co-owner code, no expiry, no throttle** `ADMN-` + 4 characters from 32 symbols = 1,048,576 combinations, stored plaintext, accepted by an anon-callable RPC with no attempt limit.	CRITICAL	CONFIRMED	`co_owner_repository.dart:9-15;` `auth_repository.dart:196-204`
C6	**"GPS-verified / tamper-proof" asserted entirely by the client** Coordinates, timestamp and the photo stamp are produced on-device. The commercial claim has no server-side trust boundary.	HIGH — TRUST	CONFIRMED	`image_stamp_service.dart; submit_report params`
H1	**"Delete account" only soft-deletes; hard-delete never invoked** Flags are set and the user is signed out; `delete-account` is never called from the app.	HIGH — LEGAL	CONFIRMED	`auth_repository.dart:183-193;` `profile_edit_screen.dart`
H2	**Privacy policy is an unfilled template and contradicts the app** Placeholders remain, and it states data is not used for third-party advertising while AdMob ships.	HIGH — LEGAL	CONFIRMED	`Privacy Policy.txt:27-44, 89-90, 101`
H3	**Notification function callable by anyone** Uses the service role and trusts the body. Any anon-key holder can trigger pushes.	HIGH	CONFIRMED	`send-notification/index.ts:15-27`
N3	**Notification function trusts `project_id` independently of `submission_id`** The mute check and deep-link route use the caller-supplied project, so muting can be bypassed. The 404/200 difference is a submission-existence oracle.	HIGH	NEW	`send-notification/index.ts:21, 49-54, 104`
N4	**Plan limits enforced only in the UI** User-per-project cap is a client-side count; `freePlanMaxSubmissions` is referenced nowhere. Direct PostgREST inserts bypass both.	HIGH — COST	NEW	`project_user_repository.dart:58-71;` `app_constants.dart:44`

ID	FINDING	SEVERITY	STATUS	EVIDENCE
H4	**Digital goods sold through Razorpay inside the app** Google Play requires Play Billing; Apple requires IAP. Removal risk as shipped.	HIGH — STORE	CONFIRMED	`billing_screen.dart:103-125`
H5	**Background location declared but never used** Only `getCurrentPosition` is called. Play needs a declaration and video; Apple rejects unused "Always".	HIGH — STORE	CONFIRMED	`AndroidManifest.xml:9; ios/Runner/Info.plist:35`
H6	**Owner session stored unencrypted; backups enabled** `flutter_secure_storage` is declared and never used (0 references). No `allowBackup` flag.	HIGH	CONFIRMED	`pubspec.yaml; AndroidManifest.xml:26-30`
H7	**Free-tier submission cap not enforced** `freePlanMaxSubmissions = 50` is dead code. No throttling anywhere.	HIGH — COST	CONFIRMED	`app_constants.dart:44`
N5	**No gitignore entry for signing material** `key.properties` and `*.jks` are unmatched, so the C2 fix will be committed by default.	MEDIUM	NEW	`.gitignore (no key/secret rules)`
N6	**Firebase browser key unrestricted** Public by design, but without referrer restriction and App Check it can be used to burn quota.	MEDIUM	NEW	`web/index.html:28; firebase-messaging-sw.js:13`
M1	**Profile update writes NULL into omitted columns**	MEDIUM	CONFIRMED	`auth_repository.dart:146-157`
M2	**Image upload failures swallowed** A "successful" report can contain zero photos.	MEDIUM	CONFIRMED	`submission_repository.dart:184-186, 239-241`
M3	**Co-owner can reach the billing route**	MEDIUM	CONFIRMED	`app_router.dart:45-98`
M4	**Deletion never removes Storage objects**	MEDIUM	CONFIRMED	`delete-account/index.ts:65-91`
M5	**No server-side OTP send throttling**	MEDIUM	CONFIRMED	`confirm_email_screen.dart:26-62`
M6	**No iOS privacy manifest**	MEDIUM	CONFIRMED	no `PrivacyInfo.xcprivacy`

ID	FINDING	SEVERITY	STATUS	EVIDENCE
N7	**Edge Functions echo internal errors to the caller**	LOW	NEW	`send-notification:136; delete-account:95`
N8	`requestLegacyExternalStorage` **enabled** Opts out of scoped storage on Android 10.	LOW	NEW	`AndroidManifest.xml:30`

Status: *New* — found in this pass. *Confirmed* — re-verified at 22cb38d, unchanged. *Upgraded* — previously reported, now understood to be more severe.

1 · Detail on the new findings

N1 — No Razorpay order, client-chosen amount

```
// lib/features/owner/billing/screens/billing_screen.dart
final options = {
  'key': AppConstants.razorpayKeyId,
  'amount': _totalPaise,          // line 28: _extraProjects * _pricePerProject
  'name': AppConstants.appName,
  // no 'order_id'
};
_razorpay.open(options);
```

Razorpay returns `razorpay_signature` only for checkouts opened against a server-created **order**. With no order, the success callback proves only that the SDK reached a success state. Even a correct `verify-payment` function added later could not validate these payments, because there is no `order_id` to include in the HMAC.

Impact: a modified client pays an arbitrary amount (including ₹0 via a tampered callback) and is then entitled to grant itself Pro, because C1 lets it write the entitlement columns.

Fix: create the order server-side (`POST /orders` with the price from a server table), return `order_id` to the app, pass it to checkout, and grant entitlement only from a function that verifies `HMAC_SHA256(order_id|payment_id, key_secret)` and writes the row with the service role. Revoke `UPDATE` on the billing columns from `authenticated` .

N2 — Deletion deletes the login, keeps the data

```
// supabase/functions/delete-account/index.ts
for (const table of tables) {
  try {
    await admin.from(table).delete().eq('owner_id', ownerId);   // returns { data, error }
  } catch (_) {                                                 // never runs
    // ignore and continue
  }
}
// ...
const { error: delErr } = await admin.auth.admin.deleteUser(ownerId);
```

`supabase-js` resolves failed PostgREST calls with `{ error }` ; it does not throw. The `catch` is therefore unreachable and every failure is discarded. The auth user is removed on line 88 regardless, so the account can no longer be signed into to retry.

The table list also deletes `submission_images` by `owner_id` . Images are keyed by `submission_id` ; if that column does not exist, the delete fails on every call and is ignored.

Impact: the user believes their data is gone, the policy says it is gone, and every submission and photo URL stays live. This is a DPDP Act exposure, not a cosmetic bug.

Fix: check `error` on each call and abort on failure; delete children by `submission_id` (join through `submissions`); remove Storage objects; delete the auth user last and only after everything else succeeded. Return a machine-readable status instead of a bare `{ success: true }`.

N3 — Notification function trusts two unrelated identifiers

```
const { submission_id, project_id, project_user_id } = await req.json();   // line 21
// mute check uses the caller-supplied project:
.eq("project_id", project_id)                                              // line 53
// deep link built from the same value:
route: `/owner/project/${project_id}`                                      // line 104
```

Nothing ties `project_id` to the submission row. An anon caller can therefore pass a project that is not muted and bypass the owner's mute setting, or walk submission IDs and use the 404-vs-200 difference to confirm which submissions exist.

Fix: read `project_id` and `owner_id` from the submission row only; require proof of the field user's access code (or a short-lived token minted by `submit_report`) before sending; rate-limit per submission and per owner; return a uniform response for unknown IDs.

N4 — Plan limits live only in the UI

```
// lib/repositories/project_user_repository.dart
final existingActive = await supabase.from('project_users')
    .select('id').eq('project_id', projectId).eq('is_active', true);
if ((existingActive as List).length >= AppConstants.freePlanMaxUsersPerProject) { ... }
```

This is a client-side count. Any holder of the anon key can insert `project_users` rows directly. `freePlanMaxSubmissions` is not referenced anywhere in `lib/`.

Fix: enforce in a trigger or in the RPC that creates users and submissions, counting rows server-side.

N5 / N6 / N7 / N8

- **N5** — `.gitignore` contains no `key.properties` or `*.jks` rule. Add both, plus `*.keystore`, before doing the C2 fix.
- **N6** — the Firebase browser key is identical in `web/index.html` and `web/firebase-messaging-sw.js`. Restrict it to your HTTP referrers in Google Cloud Console and turn on App Check.
- **N7** — `send-notification:136` returns `error.message` and `delete-account:95` returns `String(e)`. Log internally, return a generic message.
- **N8** — `android:requestLegacyExternalStorage="true"` disables scoped storage on Android 10. Remove it; nothing in the app needs shared external storage.

2 · Automated scan attempt (and why the result is not evidence of safety)

A Strix 1.7.0 scan was run against a clean worktree of `22cb38d` with Docker isolation. Five attempts were made across three providers. **None produced a usable report.** The "0 vulnerabilities" output from these runs must not be read as a clean bill of health — in each case nothing was actually tested.

MODEL	OUTCOME	CAUSE
`gemini-2.5-flash`	Model refused	Model retired for new API keys.
`gemini-3.5-flash`	Quota exhausted	Free tier allows 20 requests per model per day; a scan needs hundreds.
`mistral-large-4`	HTTP 429 on first call	Free tier caps at 20,000 tokens/minute. A single Strix prompt exceeds that ceiling.
`codestral-latest`	Completed, 0 findings	625K tokens/minute available, but the model produced 4.7K output tokens against 3.2M input — it read files without reasoning. Agents recorded no coverage.
`ministral-14b-latest`	Max turns, 0 findings	Same pattern on a narrowed three-area scope.

Interpretation. An agentic scanner needs a model that can hold a large system prompt and plan across many tool calls. Free API tiers either cap requests per day (Gemini), cap tokens per minute below a single prompt (Mistral large), or route to models that do not perform agentic reasoning (Codestral, Ministral). A paid key with a real throughput allocation, or a Strix Cloud credit balance, is the only way to get a machine-validated pass.

What this does not change. The findings in §1 are code-level facts and were verified by reading the source at `22cb38d`. A scanner would not have been able to validate them further anyway: the live Supabase project was out of scope (no credentials, and probing it without consent is not acceptable), so any "proof of concept" would have been a code excerpt either way.

3 · Required before launch — in order

1. **Rebuild the payment flow (N1 + C1).** Server-created order, server-verified signature, server-written entitlement, column-level grants revoked. Until this exists, do not change the Razorpay key from `rzp_test_` to live.
2. **Make deletion real (N2 + H1 + M4).** Check every error, delete children and Storage objects, remove the auth user last, and call the function from the app instead of soft-deleting.
3. **Real upload keystore (C2 + N5).** `key.properties`, signing config, gitignore rules, and a Play App Signing enrolment.
4. **Version the schema and audit RLS (C3).** `supabase db pull` into `supabase/migrations/`, then run the queries in Appendix A.

5. **Private bucket and signed URLs (C4),** issued only after server-side validation of the uploader's code.
6. **8+ character expiring codes, hashed at rest, attempt lockout (C5).**
7. **Harden the notification function (H3 + N3)** and move plan limits server-side (N4 + H7).
8. **Store policy and legal (H2, H4, H5, M6).** Play Billing or external sale, remove the unused background-location permission, complete the privacy policy and Data Safety form.

Appendix A — Verification SQL (run in the Supabase SQL Editor, staging first)

Expected safe results: every table `relrowsecurity = true`; no policy granting `anon`; no `UPDATE` on billing columns; RPCs `prosecdef = true` and filtering `is_active`; bucket `public = false`.

```
-- 1. Is RLS enabled on every table?
select relname, relrowsecurity
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r'
order by 1;

-- 2. Every policy that grants the anon role
select tablename, policyname, cmd, roles::text, qual, with_check
from pg_policies
where schemaname = 'public' and 'anon' = any(roles)
order by 1, 2;

-- 3. Can an owner rewrite their own billing columns?
select grantee, privilege_type, column_name
from information_schema.column_privileges
where table_name = 'owners'
  and grantee in ('anon', 'authenticated')
  and privilege_type = 'UPDATE';

-- 4. Are the RPCs SECURITY DEFINER, and do they check is_active?
select p.proname, p.prosecdef, pg_get_functiondef(p.oid)
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname = 'public'
  and p.proname in ('login_with_code', 'login_co_owner_with_code',
                    'submit_report', 'add_submission_image',
                    'get_user_submissions', 'generate_access_code');

-- 5. Is the bucket public, and who can write?
select id, name, public from storage.buckets;
select policyname, cmd, roles::text, qual, with_check
from pg_policies
where schemaname = 'storage' and tablename = 'objects';

-- 6. Does submission_images even have an owner_id column? (N2)
select column_name from information_schema.columns
where table_schema = 'public' and table_name = 'submission_images';
```

Golden tests (staging only)

```
-- A. Entitlement tampering (C1). Run as an ordinary owner from the debug console.
update owners set plan_type = 'pro', pro_project_slots = 999 where id = auth.uid();
-- Must fail with a permission error once column-level grants are in place.

-- B. Limit bypass (N4). As an owner, insert a 6th project_user for one project.
insert into project_users (project_id, owner_id, name, access_code, is_active)
values ('<project-uuid>', auth.uid(), 'extra', 'ADMN-XXXX', true);
-- Must be rejected by a server-side trigger once N4 is fixed.

-- C. Deletion completeness (N2). Delete an account, then check for orphans.
select count(*) from submissions  where owner_id = '<deleted-owner-uuid>';
select count(*) from submission_images si
  join submissions s on s.id = si.submission_id
 where s.owner_id = '<deleted-owner-uuid>';
-- Both must be zero.
```

Severity legend

- **CRITICAL** — exploitable now; enables data breach, free entitlements, or a compliance failure that cannot be undone.
- **HIGH** — likely exploitation or a hard store/legal rejection; fix before launch.
- **MEDIUM** — real impact but needs a specific condition or an insider; fix in the first release window.
- **LOW** — hygiene; fix when convenient.

Static review of commit 22cb38d. Database policies, RPC bodies, trigger definitions, bucket flags and Razorpay webhook configuration were not available in the repository and are listed as unverified in C3, C4 and N2. No live endpoint was queried. Automated scanning was attempted and blocked by provider limits; see §2. Prepared for the SahiReport maintainers.